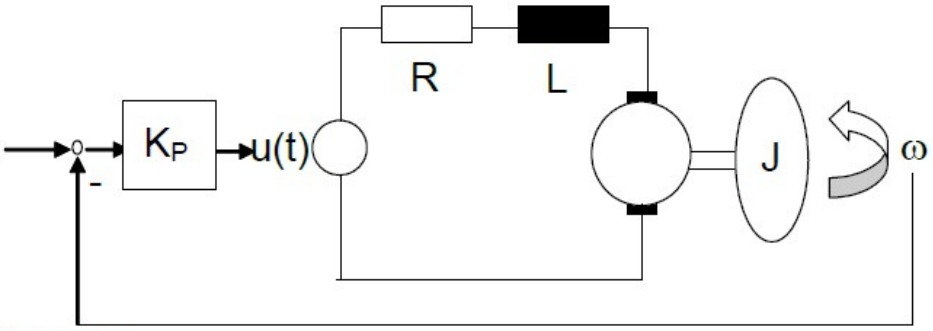
Exercise 4
Model of a speed control

Group 10
Rakesh Nuggehalli Ramesh - 15546621
Sakshi Arali - 23046143
Elrick Dsouza - 27846430
Meghana Chitteri - 10546387

Speed control is used in many application of mechatronic systems e. g. for robots. In this exercise a model of a speed control system is given. This model must be implemented using Matlab/Simulink.



The equations are:

$$u(t) = R \cdot i(t) + L \cdot \frac{d}{dt}(i(t)) + c_m \cdot \dot{\alpha}(t); \quad \leftrightarrow \quad \frac{d}{dt}(i(t)) = \frac{1}{L}(u(t) - R \cdot i(t) - c_m \cdot \omega(t))$$

$$J \cdot \ddot{\alpha}(t) = c_m \cdot i(t) - D \cdot \dot{\alpha}(t); \qquad \frac{d}{dt}(\omega(t)) = \frac{1}{J}(c_m \cdot i(t) - D \cdot \omega(t))$$

```
p.L = 0.23e-3;  %[V/A*sec]      Induktivity
p.cm = 23.4e-3; %[Nm/A]         Motor-constant
p.R = 2.4;      %[Ohm]          Resistanace
p.J = 0.23e-6;  %[N*m^2]        Inertiay
p.D = 0.4191e-5;%[Nm/rad*sec]   Damping
```

Figure 1:

a) Build up a simulation model of the dc-motor (open-loop) and test it!

.

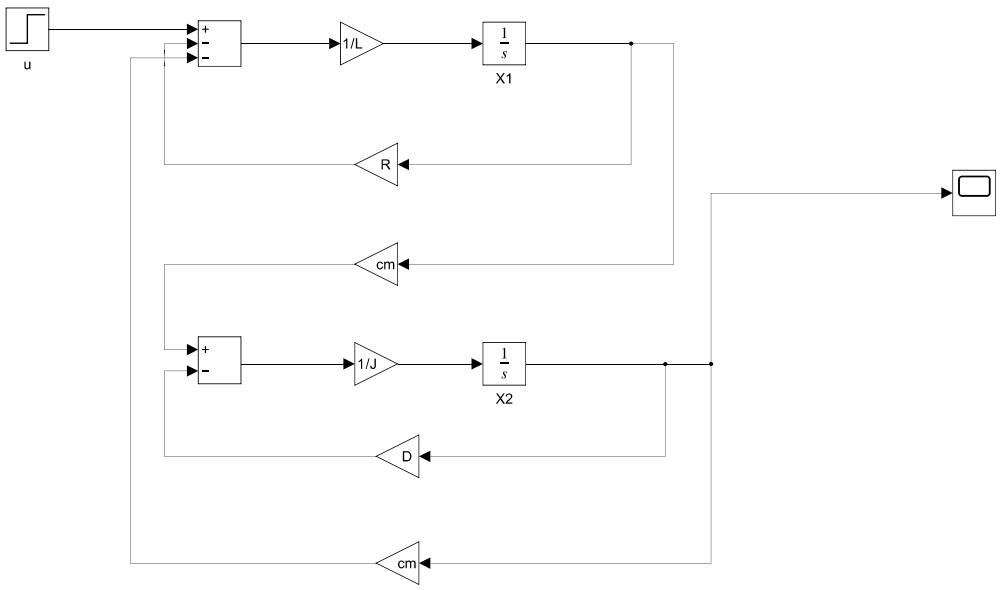


Figure 1:

This Simulink diagram is a state-space model of a DC motor, split cleanly into its electrical and mechanical dynamics. Let's walk through it block by block and connect it to the physics. Mechanical subsystem (bottom loop)

This part models rotational motion.

```
1
2 %% Excercise 4a
3 clear;clc;clf
4 L = 0.23e-3; %[V/A*sec] Induktivity
5 cm = 23.4e-3; %[Nm/A] Motor-constant
6 R = 2.4; %[Ohm] Resistanace
7 J = 0.23e-6; %[N*m^2] Inertiay
8 D = 0.4191e-5;%[Nm/rad*sec] Damping
9
10 out = sim('Excercise4a.slx');
11
12 plot(out.ex4a.time, out.ex4a.signals(1).values);
13 xlabel('Time [sec]');      % Label x-axis as time in seconds
14 ylabel('Speed '); % Label y-axis as speed in radians per second
15 ylabel('Speed [omega]'); % Label y-axis as speed in radians per
      second with ohmega symbol
16 grid;                     % Enable grid for better visualization
17 title('Step Response of a DC Motor');
```

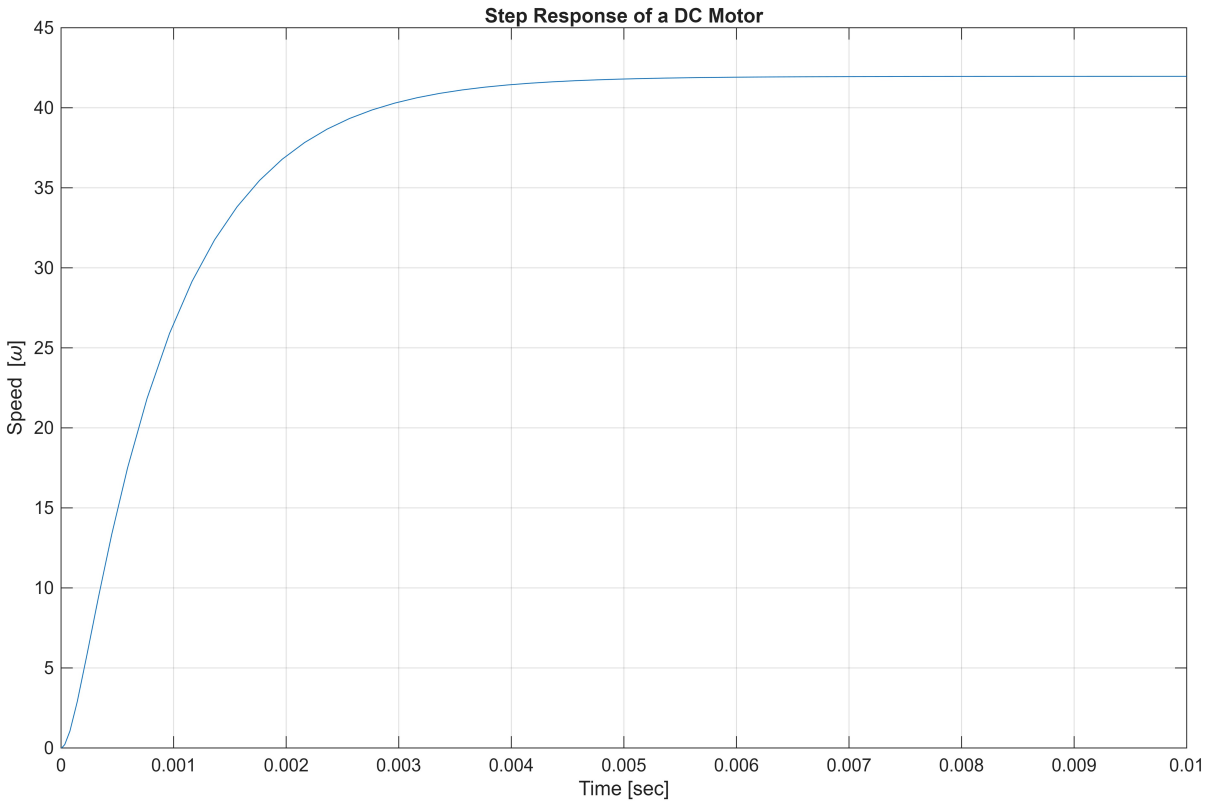


Figure 2:

This plot shows the step response of a DC motor, specifically how the angular speed (w) changes when a step input (usually a sudden constant voltage) is applied. Graph explained-

1. At t = 0

   A step input is applied (e.g., voltage suddenly turns on).
   Motor speed starts at 0 because of inertia.

   2. Rapid acceleration (0– 0.002 s)
   Speed rises steeply.
   This is the motor overcoming inertia and friction.
   Electrical dynamics dominate early on.

   3. Slowing rise (0.002–0.005 s)
   The curve starts bending (concave down).
   Back EMF increases as speed increases, reducing net torque.

   4. Steady-state (0.005 s onward)
   Speed settles around 42 rad/s.
   Input torque = load + losses.

No oscillations → stable and well-damped system.

b) Change the parameters of the controller to get a good control behavior!

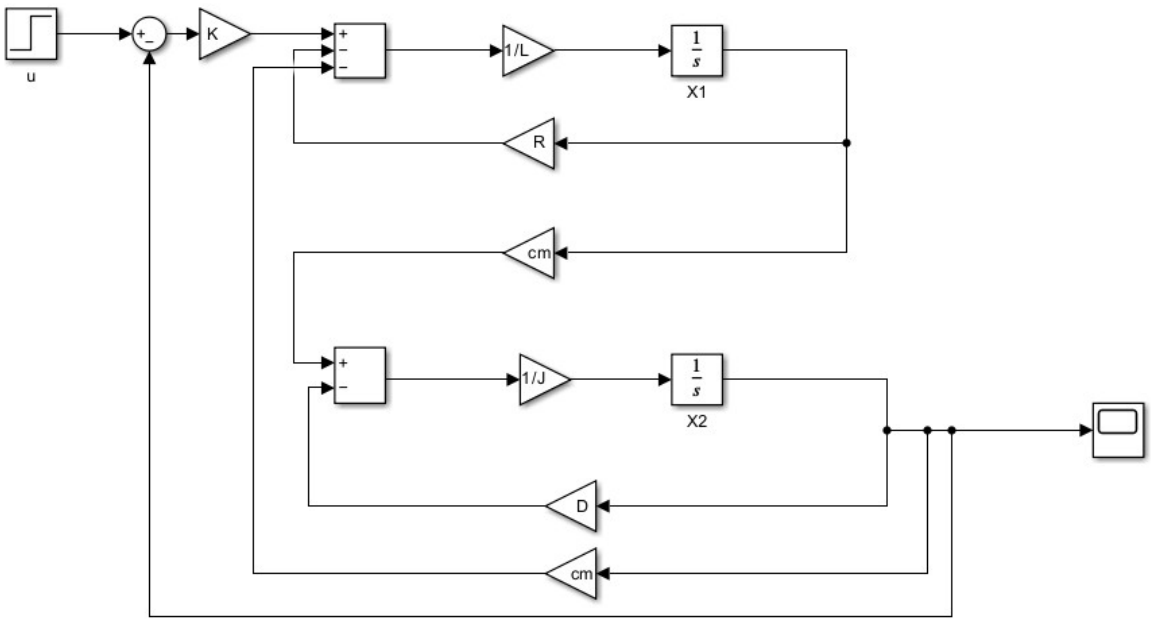


Figure 3:

The new element is the gain K, which closes a speed-control loop. A feedback loop from speed () is added to the input.A gain K acts as a controller.

The motor is now controlled, not just open-loop.

So this is a closed-loop DC motor speed control system. Summary of the Simulink model:

The model represents a closed-loop DC motor speed control system.

The motor has two states: X1: armature current X2: angular speed The reference input u is the desired speed.

Actual speed is fed back and compared with u to form an error.

A proportional controller (gain K) converts this error into the motor's input voltage. The electrical subsystem models current dynamics using L and R, with back-EMF (cm·) feedback. The mechanical subsystem models speed dynamics using inertia J and damping D, with torque (cm·i). The motor constant cm couples electrical and mechanical dynamics. Feedback forces the motor speed to track the reference, improving accuracy and response speed.

```
    %% Excercise 4b
clear;clc;clf
L = 0.23e-3; %[V/A*sec] Induktivity
cm = 23.4e-3; %[Nm/A] Motor-constant
R = 2.4; %[Ohm] Resistanace
J = 0.23e-6; %[N*m^2] Inertiay
D = 0.4191e-5;%[Nm/rad*sec] Damping
K = 0.20;%Gain

out = sim('Excercise4a');

plot(out.ex4a.time, out.ex4a.signals(1).values);
xlabel('Time [sec]');      % Label x-axis as time in seconds
ylabel('Speed '); % Label y-axis as speed in radians per second
ylabel('Speed [\omega]'); % Label y-axis as speed in radians per
   second with ohmega symbol
grid;                      % Enable grid for better visualization
title('Step Response of a DC Motor');
```

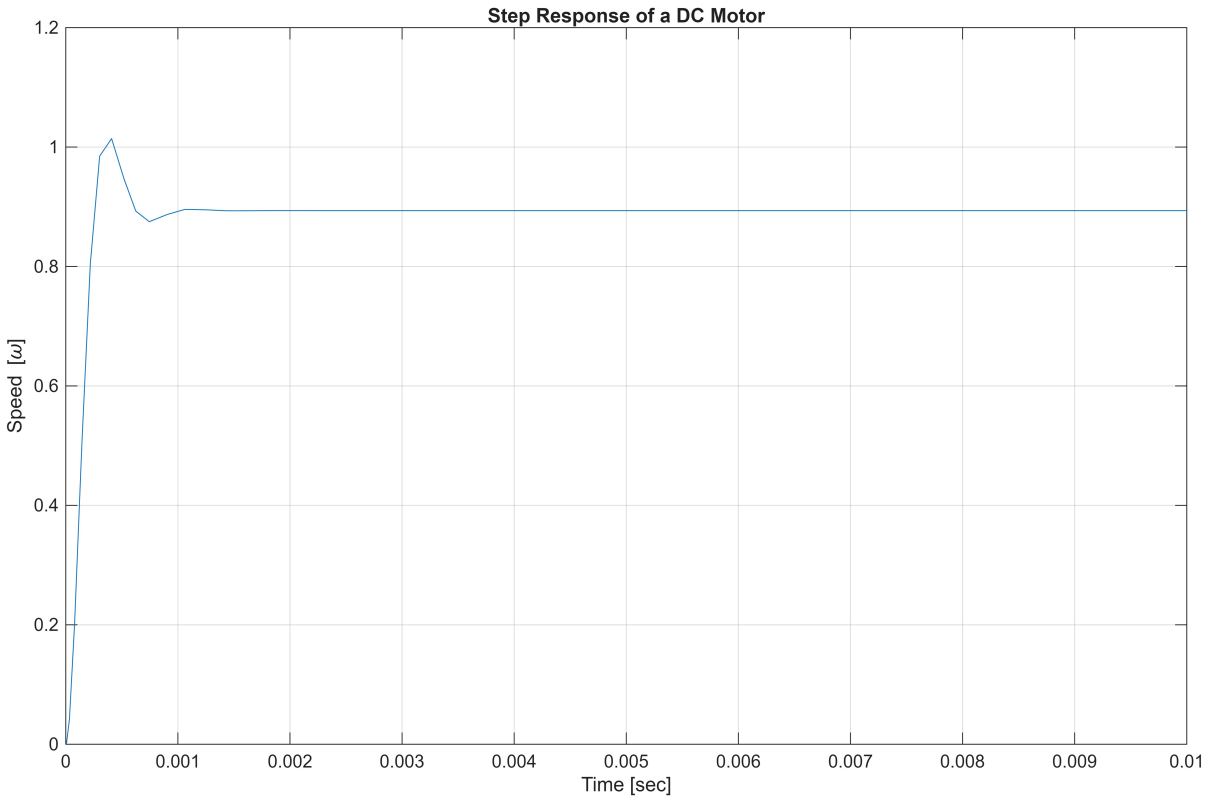


Figure 4:

This graph shows the step response of a DC motor speed over a very short time window (0–0.01 s): The motor speed rises very rapidly from 0, indicating a fast system. There is a small overshoot early on (peaks slightly above 1.0). After the overshoot, the speed drops and settles smoothly. The system reaches a steady-state speed around 0.89–0.9. Settling time is very short (on the order of milliseconds). Overall behavior suggests a stable, lightly underdamped system with good damping and quick response.

c) Try different solvers (use additionally solvers for stiff systems) and compare the results! Plot the simulated motor-current, calculated with different solver in the same plot!

```
   %% Exercise 4c     Solver comparison
clear; clc; clf

% Motor parameters
L  = 0.23e-3;      % [H]
cm = 23.4e-3;      % [Nm/A]
R  = 2.4;          % [Ohm]
J  = 0.23e-6;      % [Nm^2]
D  = 0.4191e-5;    % [Nm*s/rad]
K  = 0.20;         % Controller gain

model = 'Excercise4a';

% Define solvers to test
solvers = {'ode45','ode23','ode15s'};
titles  = {'ode45 (non-stiff)', ...
           'ode23 (low order)', ...
           'ode15s (stiff solver)'};

figure
for k = 1:length(solvers)

    % Set solver
    set_param(model, 'Solver', solvers{k});

    % Run simulation
    out = sim(model);

    % Extract motor current (adjust signal index if needed)
    t = out.ex4a.time;
    i = out.ex4a.signals(1).values;

    % Plot in subplot
```

```
    subplot(length(solvers),1,k)
    plot(t,i,'LineWidth',1.2)
    grid on
    ylabel('Current [A]')
    title(titles{k})

    if k == length(solvers)
        xlabel('Time [s]')
    end
end

sgtitle('Motor Current     Comparison of Different Solvers')
```

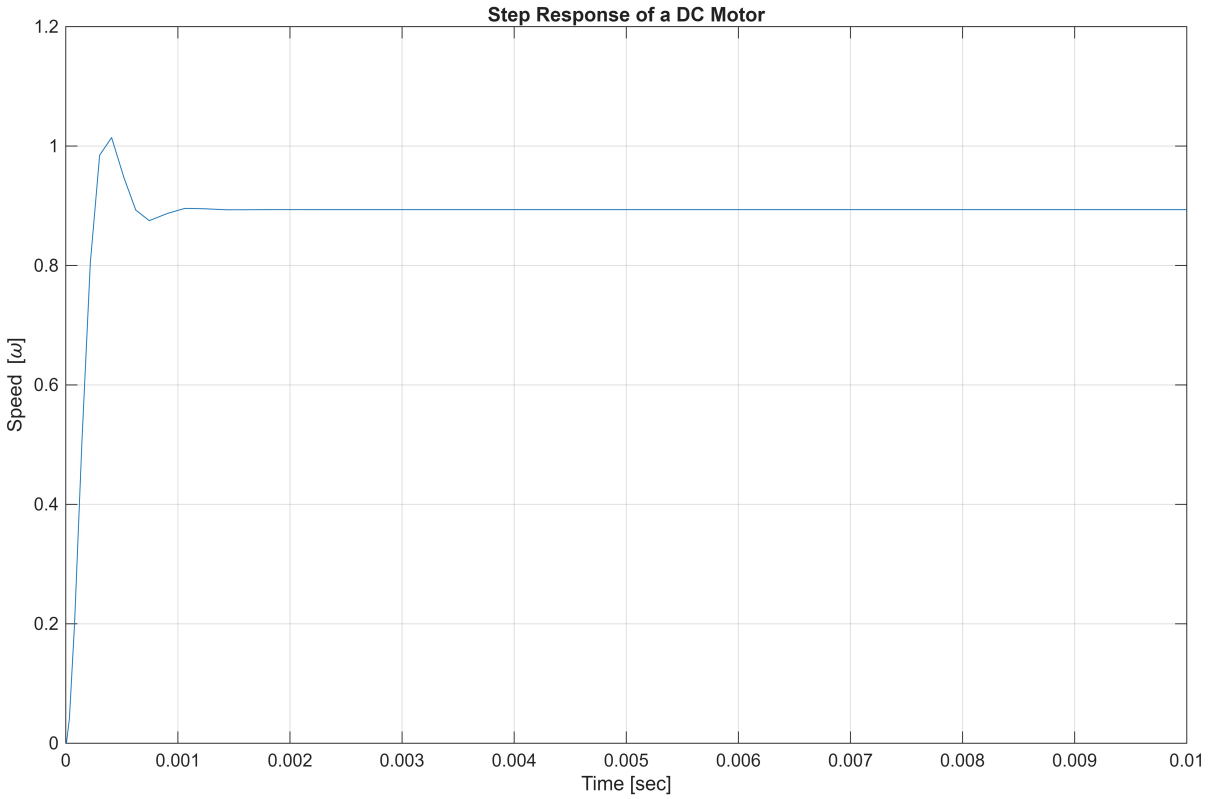


Figure 5:

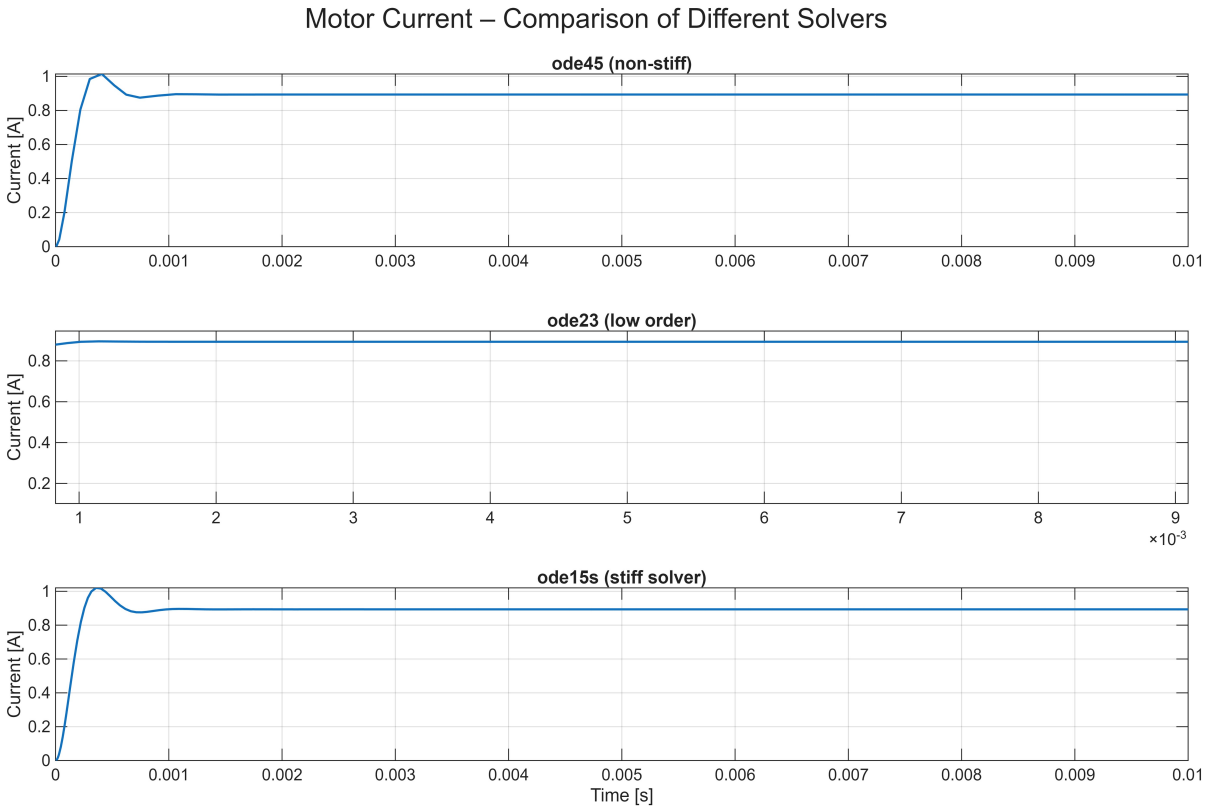


Figure 6:

This plot compares the motor current step response computed using three different ODE solvers:

ode45 (non-stiff): Fast rise in current with a noticeable initial overshoot (near 1 A), followed by a quick decay and settling around 0.88–0.9 A.

ode23 (low-order): Produces a much smoother response with little to no visible overshoot. It reaches approximately the same steady-state current but misses some fast transient details.

ode15s (stiff solver): Very similar to ode45, capturing the sharp transient and overshoot accurately, then settling to the same steady-state value.

Overall takeaway: All solvers converge to the same steady-state current, but ode45 and ode15s better capture fast transient dynamics, while ode23 smooths out rapid changes, making it less accurate for sharp electrical transients.